Curs 13

9.01.2008

12. Fişiere

Fişiere

- ✓ În cazul unui program care conţine mai multe fişiere sursă unde o variabilă este definită în fişierul 1 şi folosită în fişierul 2 atunci e nevoie de o declaraţie *extern* în fişierul 2 pentru a conecta cele 2 apariţii ale variabilei.
- ✓ Definiţia unei variabile externe se referă la locul în care variabila este creată şi i se alocă memorie; declaraţia se referă la precizarea variabilei, fără a i se aloca memorie.

Fişiere

- ✓ Atunci când este executat un program, sunt deschise automat 3 fişiere şi sunt furnizaţi *pointerii* asociaţi cu acestea.
- ✓ Aceste fişiere sunt: intrarea standard, ieşirea standard, ieşirea standard pentru semnalarea erorilor (***stdin, stdout, stderr***).

Fişiere – deschidere fişier

- ✓ Înainte de a citi sau scrie din/într-un fişier, acesta trebuie deschis cu ajutorul funcţiei ***fopen*** din biblioteca standard.
- ✓ *fopen* ia un nume extern (ex. fis1.c sau fis2.c), realizează câteva acţiuni legate de sistemul de operare şi returnează un nume intern care va fi utilizat ulterior la citirile şi scrierile conexe fişierului.

Fişiere – deschidere fişier

- ✓ Acest nume intern este un *pointer*, numit ***file pointer***, la o structură care conţine informaţii despre fişiere: adresa unui *buffer*, poziţia curentă a unui caracter în *buffer*, un indicator care precizează dacă acţiunea este citire sau scriere în fişier, etc.
- ✓ În biblioteca standard de I/O, *stdio.h*, este definită structura FILE (tipul şi prototipurile funcţiilor de prelucrare a fişierelor).

Fişiere – deschidere fişier

- ✓ Singura declaraţie necesară pentru un *pointer* de fişier este:

```
FILE *fopen(), *fp;
```

  - *file* este un nume de tip (este implementat cu *typedef*).
  - *fopen* returnează un pointer la *file*.
  - fp este un pointer la *file.*

- ✓ Funcţia ***fopen*** returnează un *pointer* spre tipul FILE în caz de succes sau *pointer*ul nul în caz de eroare.
- ✓ Apelarea lui *fopen* în program:

```
fp = fopen(nume, mod);
```

Fişiere – deschidere fişier

- ✓ nume – este un *pointer* spre un şir de caractere care defineşte calea spre fişier;
- ✓ mod – este un *pointer* spre un şir de caractere care defineşte modul de prelucrare pentru fişierului deschis:
  - r - deschidere fişier text pentru citire;
  - w - crearea unui fişier text pentru scriere; dacă fişierul indicat există, conţinutul său va fi rescris cu noile informaţii;
  - a - deschidere fişier text pentru adăugare la sfârşitul fişierului sau crearea unui fişier pentru scriere în cazul în care acesta nu există încă;
  - r+ - deschidere fişier text pentru actualizare (citire şi scriere);
  - w+ - crearea unui fişier pentru actualizare;
  - a+ - deschidere pentru actualizare la sfârşitul fişierului;
  - Pentru a preciza faptul că un anumit fişier este deschis sau creat în mod text se adaugă 't' la şirul mod obişnuit (rt, w+t, etc.)
  - Pentru a preciza faptul că un anumit fişier este deschis sau creat în mod binar se adaugă 'b' la şirul mod obişnuit (wb, r+b, etc.)
  - Ex. : r+t este echivalent cu rt+
  - Dacă nu este indicat unul dintre modurile text sau binar, modul este dat de _*fmode*: O_BINARY sau O_TEXT (cele două constante sunt definite în fcntl.h)

Fişiere – deschidere fişier

- ✓ Conţinutul unui fişier care există deja, deschis în scriere în mod ‘w’ , va fi şters (este considerat deschis în creare).
- ✓ Deschiderea unui fişier pentru adăugare (mod ‘a’) va avea ca efect crearea unui fişier (dacă acesta nu există) sau datele vor fi scrise la sfârsitul său (dacă există deja).
- ✓ Un fişier care nu există încă şi este deschis în mod ‘w’ sau ‘a’, va fi creat.

Fişiere – citire/scriere

- ✓ Un fişier odată creat va fi folosit pentru a scrie în el informaţie, iar apoi se vor realiza prelucrări cu conţinutul său. Prelucrările pot fi:
  - pe caracter (*getc, putc*);
  - pe şiruri de caractere (*fgets, fputs*);
  - cu format (*fscanf, fprintf*).

getc/putc

- ✓ *getc* returnează următorul caracter dintr-un fişier

  c = getc(fp)

Are ca efect plasarea următorului caracter din fişierul referit prin fp în c şi *EOF* când se ajunge la sfârşitul fişierului.

- ✓ *putc*

  putc(c, fp) plasează caracterul c în fişierul referit prin fp şi returnează caracterul c.

- ✓ La fel ca şi *getchar* şi *putchar*, *getc* şi *putc* pot fi macrouri în loc de funcţii.

fgets/fputs

fgets (line, MAXLINE, fp)

are ca efect citirea următoarei linii de la intrare (inclusiv *newline*) din fişierul referit prin fp în tabloul de caractere line; vor fi citite cel mult MAXLINE-1 caractere. Linia rezultată este terminată prin \0.

- ✓ *fgets* returnează line, iar la sfârşitul fişierului returnează *null*.

fputs (line, fp)

- ✓ Funcţia *fputs* scrie un şir (care nu conţine neaparat *newline*) într-un fişier

fscanf/fprintf

- ✓ Sunt similare cu *scanf* şi *printf*, cu diferenţa că primul argument este un *pointer* fişier care indică spre fişierul care va fi citit sau scris; şirul de control este al doilea argument.

Închidere fişier

- int **fclose** (FILE *stream);

- Închide fluxul indicat (*stream*). Toate *buffer*ele asociate cu fluxul sunt golite înainte de închidere.
- În caz de succes returnează 0, iar în caz de eroare, *eof*.

Închidere fişier - *fclose*

- ✓ Are ca efect întreruperea legăturii dintre *pointerul* **care indică spre** fişier şi numele extern al fişierului care a fost stabilită prin ***fopen***, eliberând *pointerul* fişierului pentru un alt fişier.
- ✓ Se recomandă închiderea fişierelor după ce nu mai sunt folosite deoarece cele mai multe sisteme de operare limitează numărul de fişiere deschise simultan.
- ✓ Un alt motiv pentru a utiliza ***fclose***, este golirea *buffer*-ului în care ***putc*** colectează ieşirea.
- ✓ ***fclose*** este apelat automat pentru fiecare fişier deschis, atunci când programul se termină normal.

Precizare erori - *ferror*

int ferror (FILE *stream)

- ✓ ***ferror*** este un macro care testează dacă ă a avut loc o eroare la citire sau scriere pe un anumit flux.
- ✓ Returnează o valoare diferită de zero în cazul în care s-a depistat eroare pe fluxul indicat.

Repoziţionarea pointerului la fişier

- ***fseek*** realizează repoziţionarea *pointer*ului la fişier

  int fseek (FILE *stream, long offset, int whence)

(pentru detalii v. Help Turbo C)

- În caz de succes returnează zero, iar în caz contrar o valoare diferită de zero.

Alte funcţii

- ***ftell*** returnează *pointer*ul fişierului curent.
- ***fgetpos*** memorează locaţia *pointer*ului fişier asociat cu fluxul curent.
- ***fsetpos*** setează *pointer*ul fişier asociat cu fluxul curent la o nouă poziţie.

Exemple

- ✓ PC12_1.c
- ✓ PC12_2.c

PC Lint

Lint

- ✓ scamă
- ✓ *a soft fleecy material made from linen usually by scraping; fuzz consisting especially of fine ravelings and short fibers of yarn and fabric.*

- ✓ PC-Lint este un pachet software care permite punerea în evidenţă a
  - erorilor
  - inconsistenţelor
  - entităţilor neportabile
  - codului redundant
  - unor surse de probleme

din programele scrise în limbaj C (K/R sau ANSI C). Sau C++.

PC Lint

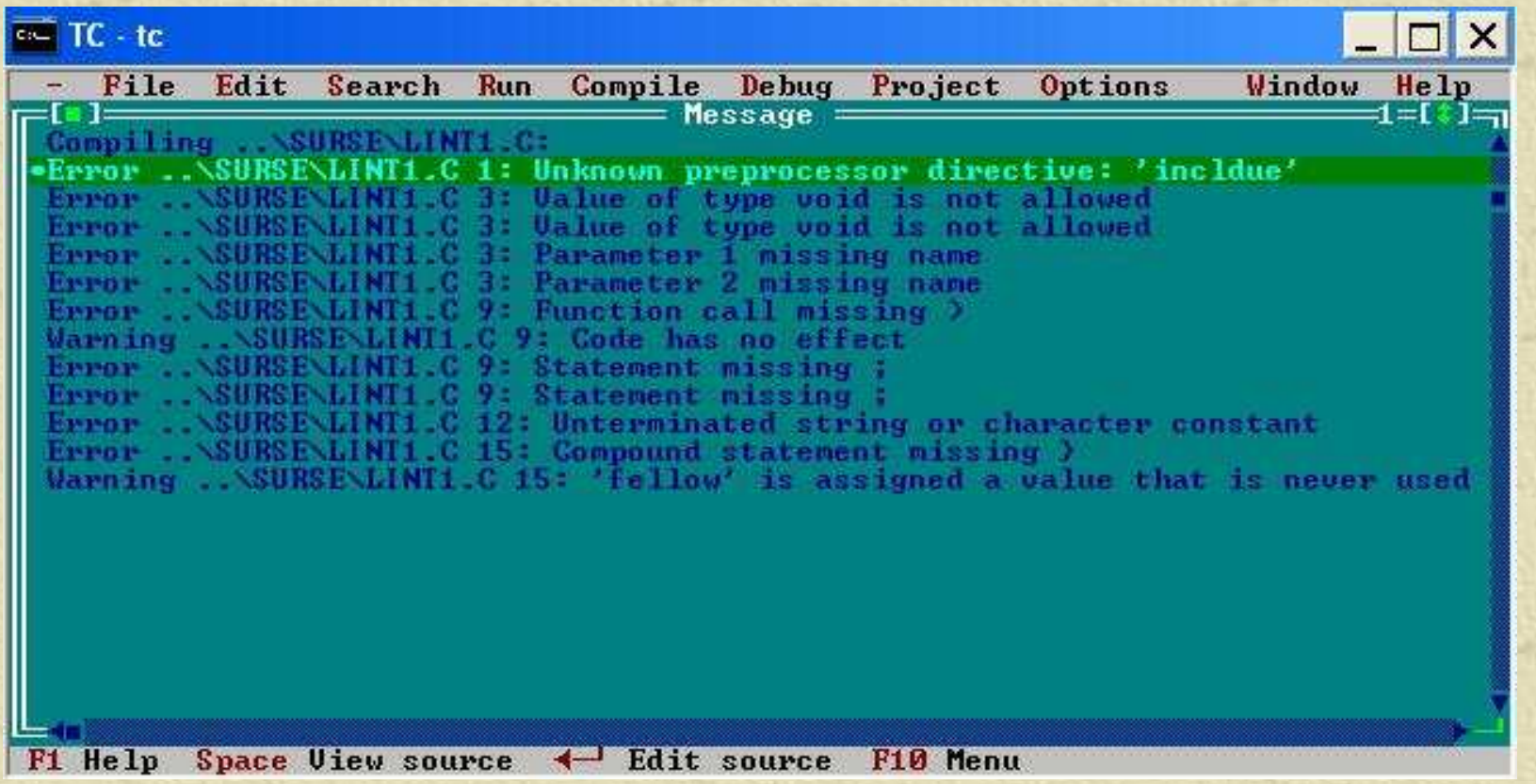
- ✓ are facilități sporite (față de compilator și link-editor) de detectare a erorilor programelor scrise în C și C++.

- ✓ Este util pentru **proiecte mari** dezvoltate în medii asociate cu **C și C++** (peste 100 000 linii sursă).
  - un programator mediu are între 5 și 10% erori, iar 1/2 din timpul de dezvoltare pentru proiecte mari se pierde cu depanarea acestuia.
  - În acest sens facilitățile oferite de PC Lint ar **scurta timpul de dezvoltare**. De asemenea, se obțin produse-program mai **stabile, robuste**.

- ✓ PC Lint în conexiune cu **embedded systems** - dezvoltatori în domeniu care folosesc și recomandă acest instrument software.

PC Lint

- ✓ **lint1.c**
  - 10 erori, 2 atenţionări



PC Lint

✓ Keil - 26 erori

```
--- Module:   badcode.c
badcode.c  10  Error 16:
               Unrecognized name
badcode.c  10  Error 10:
               Expecting end of line
badcode.c  12  Error 66:
               Bad type
badcode.c  12  Error 66:
               Bad type
badcode.c  19  Info 718:
               fer undeclared, assumed to return int
badcode.c  19  Info 746:
               call to fer not made in the presence of
               a prototype
badcode.c  19  Error 10:
               Expecting ','
badcode.c  19  Error 26:
               Expected an expression, found ';'
badcode.c  19  Warning 522:
               Expected void type, assignment,
               increment or decrement
badcode.c  19  Error 10:
               Expecting ';'
badcode.c  19  Error 10:
               Expecting ';'
badcode.c  21  Info 718:
               printf undeclared, assumed to return int
badcode.c  21  Info 746:
               call to printf not made in the presence of
               a prototype
```

```
badcode.c  23  Info 737:
               Loss of sign in promotion from long to
               unsigned long
badcode.c  23  Info 713:
               Loss of precision (assignment) (unsigned
               long to long)
badcode.c  24  Error 2:
               Unclosed Quote
badcode.c  25  Error 10:
               Expecting ','
badcode.c  26  Error 10:
               Expecting ','
badcode.c  26  Error 26:
               Expected an expression, found '}'
badcode.c  26  Warning 559:
               Size of argument no. 2 inconsistent with
               format
badcode.c  26  Warning 516:
               printf has arg. type conflict (arg. no. 2
               -- pointer vs. unsigned int) with line 21
badcode.c  27  Warning 550: fellow (line 15) not accessed

--- Global Wrap-up

Warning 526: printf (line 21, file badcode.c) not defined
Warning 628: no argument information provided for function
             printf (line 21, file badcode.c)
Warning 526: fer (line 19, file badcode.c) not defined
Warning 628: no argument information provided for function
             fer (line 19, file badcode.c)
```

PC Lint

- ✓ Verifică compatibilitatea tipurilor, pierderi de precizie, compatibilităţi între module, prototipul funcţiilor, entităţile neutilizate, cod redundant, şamd.
- ✓ Iniţial zeci de tipuri de erori şi atenţionări – în prezent sute … - organizate pentru a putea fi personalizată afişarea (*header*, număr, etc.)

Examinare

- ✓ **Test grilă** (eliminator): 16.01.2007, ora 10.00 sala A106;

- ✓ **Test pe calculator**: 28, 29, 30.01.2008, Sala 5 de la CC (sala de laborator) sau B019 conform afişării; lista cu programarea **pe date şi ore** va fi afişată la avizierul sălii B019 vineri, 25.01.2008.

Examinare

- ✓ Pune în evidenţă
  - modul în care **cunoaşteţi elementele de bază ale programării structurate** prezentate la curs şi laborator (cu exemplificare în C); v. afiş.
  - cum reacţionaţi în **luarea deciziilor**;
  - cum stăpâniţi **limbajul tehnic** asociat;
  - cum faceţi **legături** între subiectele primite şi cunoştinţele acumulate pe parcurs.

Succes!